Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 10

Validare și securitate de bază

Mutăm regulile pentru recenzii în entitate, cu Symfony Validator, afișăm erorile în formular și discutăm riscurile cele mai frecvente ale unei aplicații web.

Tema	Tema 10 din planul disciplinei (examene.pe.ub.ro): securitatea aplicațiilor web
Durata	2 ore
Aveți nevoie de	API-ul și componenta `ReviewSection` din laboratoarele 08–09, Postman
La final aveți	validare cu atribute pe `Review`, erorile afișate lângă câmpuri, o demonstrație de hash pentru parole și un commit pe branch-ul vostru

1. Obiective

La finalul laboratorului veți putea:

- recunoaște trei riscuri din OWASP Top 10 și explica cum le evită aplicația noastră;
- declara reguli de validare cu atribute `#[Assert\...]` și le verifica cu `ValidatorInterface`;
- afișa în React, lângă fiecare câmp, erorile trimise de server;
- explica de ce parolele se păstrează doar ca hash și secretele doar în `.env.local`.

2. Pe scurt: OWASP Top 10 pentru începători

OWASP Top 10 este lista celor mai frecvente categorii de vulnerabilități ale aplicațiilor web, actualizată periodic de comunitatea OWASP [2]. Regula de bază: **orice dată venită de la client poate fi greșită sau rău intenționată**, chiar dacă formularul nostru o verifică deja. Validarea din browser este pentru confortul utilizatorului; validarea din server este pentru siguranță [1].

Risc	Exemplu de atac	Cum ne apărăm
Injecție SQL	în câmpul de căutare se scrie `' OR '1'='1`, iar textul ajunge lipit într-o interogare SQL	Doctrine trimite valorile ca **parametri**, separat de textul SQL (`findBy()`, `setParameter()`)
XSS (*Cross-Site Scripting*)	un comentariu conține `<script>…</script>` și rulează în browserul altor vizitatori	React **escapează** textul din `{…}`; nu folosim `dangerouslySetInnerHTML` cu date de la utilizatori
Date sensibile expuse	parole păstrate în clar, trafic necriptat, chei secrete în Git	`password_hash()`, HTTPS, secretele în `.env.local`

```
// GREȘIT: valoarea este lipită în textul SQL (injecție posibilă)
$sql = "SELECT * FROM review WHERE author = '" . $author . "'";
// CORECT: valoarea este trimisă separat, ca parametru
$sql = 'SELECT * FROM review WHERE author = ?';
$rows = $connection->fetchAllAssociative($sql, [$author]);
```

3. Pornirea

1. Porniți **Docker Desktop** (mesajul **Engine running**). Comenzile se scriu în terminalul editorului, în folderul `daw-template-ti-info-2026-2027`; pentru probleme, vedeți metodologia [6].
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                 # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

3. Verificați că este instalată componenta de validare (dacă apare o eroare, instalați-o cu `composer require symfony/validator`, tot prin `docker exec`):

```
docker exec -it daw-php composer show symfony/validator
```

4. Validarea cu Symfony Validator

4.1 Regulile, pe entitate

În `backend/app/src/Entity/Review.php`, adăugați importul de mai jos lângă celelalte `use`, apoi atributele `#[Assert\...]` deasupra proprietăților, ca în fragment. Restul clasei rămâne neschimbat. Structura tabelei nu se schimbă, deci **nu** este nevoie de migrare [3].

```
use Symfony\Component\Validator\Constraints as Assert;

  #[ORM\Column(length: 100)]
  #[Assert\NotBlank(message: 'Autorul este obligatoriu.')]
  #[Assert\Length(max: 100, maxMessage: 'Cel mult {{ limit }} caractere.')]
  private ?string $author = null;

  #[ORM\Column]
  #[Assert\Range(min: 1, max: 5,
    notInRangeMessage: 'Nota trebuie să fie între {{ min }} și {{ max }}.')]
  private ?int $rating = null;

  #[ORM\Column(type: Types::TEXT, nullable: true)]
  #[Assert\Length(max: 1000,
    maxMessage: 'Comentariul are cel mult {{ limit }} caractere.')]
  private ?string $comment = null;

  #[ORM\ManyToOne]
  #[ORM\JoinColumn(nullable: false, onDelete: 'CASCADE')]
  #[Assert\NotNull(message: 'Produsul indicat nu există.')]
  private ?Product $product = null;
```

4.2 Validatorul, în controller

În `ReviewController.php`: (a) adăugați importul `use Symfony\Component\Validator\Validator\ValidatorInterface;`; (b) adăugați în constructor, după `ProductRepository $products,`, parametrul `private ValidatorInterface $validator,`; (c) înlocuiți metoda `fill()` din laboratorul 08 cu varianta de mai jos. Verificările scrise de mână dispar: regulile stau acum într-un singur loc, pe entitate.

```
// Completează recenzia din JSON și o validează; întoarce erorile (câmp => mesaj).
private function fill(Review $review, Request $request): array
{
  try {
    $data = $request->toArray();
  } catch (JsonException) {
    return ['body' => 'Corpul cererii trebuie să fie un obiect JSON valid.'];
  }

  $author = is_string($data['author'] ?? null) ? trim($data['author']) : '';
  $comment = is_string($data['comment'] ?? null) ? trim($data['comment']) : '';
  $rating = is_int($data['rating'] ?? null) ? $data['rating'] : 0; // 0 = invalid
  $review->setAuthor($author)
    ->setRating($rating)
    ->setComment($comment === '' ? null : $comment)
    ->setProduct($this->products->find((int) ($data['productId'] ?? 0)));

  $errors = [];
  foreach ($this->validator->validate($review) as $violation) {
    $errors[$violation->getPropertyPath()] = $violation->getMessage();
  }
  return $errors;
}
```

Rezultat. În Postman, `POST /api/reviews` cu `{"author": "", "rating": 7, "productId": 9999}` întoarce **400** și `{"errors": {"author": "Autorul este obligatoriu.", "rating": "Nota trebuie să fie între 1 și 5.", "product": "Produsul indicat nu există."}}`. Metodele `create()` și `update()` nu se schimbă: tot `fill()` le dă erorile.

5. Erorile, afișate în formularul React

Funcția `createReview()` din laboratorul 09 pune deja erorile pe câmpuri în `err.fields`. În `components/ReviewSection.jsx` faceți patru modificări mici:

1. Lângă celelalte stări, adăugați:
   ```
   const [fieldErrors, setFieldErrors] = useState({});
   ```
2. În `handleSubmit`, după `setError('');` adăugați `setFieldErrors({});`, iar în `catch`, după `setError(err.message);`, adăugați:
   ```
   setFieldErrors(err.fields || {});
   ```
3. Adăugați la formular atributul `noValidate`, ca browserul să nu mai blocheze trimiterea, iar regulile serverului să se vadă:
   ```
   <form onSubmit={handleSubmit} noValidate>
   ```
4. Sub fiecare câmp (`author`, `rating`, `comment`), înainte de `</label>`, afișați eroarea lui; de exemplu, pentru autor:
   ```
   {fieldErrors.author && <span className="error">{fieldErrors.author}</span>}
   ```

Rezultat. Trimiteți formularul cu numele gol: sub câmp apare „Autorul este obligatoriu.”. Lipiți un comentariu foarte lung (peste 1000 de caractere): apare mesajul pentru comentariu. Erorile dispar la următoarea trimitere reușită.

Atenție. Testul XSS: trimiteți o recenzie cu comentariul `<img src=x onerror=alert(1)>`. React o afișează **ca text**, nu o execută. Același text, pus cu `dangerouslySetInnerHTML`, ar rula codul în browserul fiecărui vizitator.

6. Parolele și secretele

6.1 Parolele nu se păstrează niciodată în clar

O parolă se salvează doar ca **hash**: un rezultat dintr-o funcție greu de inversat, cu o „sare” (*salt*) aleatoare inclusă. La autentificare nu „decriptăm” nimic, ci verificăm dacă parola introdusă produce un hash compatibil [4], [5]. Creați un controller mic de demonstrație, `src/Controller/PasswordDemoController.php`:

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\Routing\Attribute\Route;

// Doar pentru demonstrație la laborator. Nu îl păstrați în proiectul final.
final class PasswordDemoController extends AbstractController
{
  #[Route('/api/demo/parola', methods: ['POST'])]
  public function hash(Request $request): JsonResponse
  {
    $password = (string) ($request->toArray()['password'] ?? '');
    $hash = password_hash($password, PASSWORD_DEFAULT);
    return $this->json([
      'hash' => $hash,
      'parolaCorecta' => password_verify($password, $hash),
      'parolaGresita' => password_verify($password . 'x', $hash),
    ]);
  }
}
```

Rezultat. În Postman, `POST http://localhost:8080/api/demo/parola` cu `{"password": "parola123"}` întoarce un hash care începe de obicei cu `$2y$` (algoritmul bcrypt), `parolaCorecta: true` și `parolaGresita: false`. Trimiteți de două ori: hash-ul diferă, datorită sării. În aplicații cu utilizatori, Symfony face același lucru prin `UserPasswordHasherInterface`.

6.2 Secretele în .env.local, traficul prin HTTPS

- `backend/app/.env` este în Git și conține doar valori de dezvoltare. Parolele reale, cheile de API și alte secrete se pun în `backend/app/.env.local`, care **nu** ajunge în Git și are prioritate față de `.env` [3].
- Verificați că `.env.local` este ignorat. Dacă nu se afișează nimic, adăugați rândul `/.env.local` în `backend/app/.gitignore`:

  ```
  git check-ignore -v backend/app/.env.local
  ```

- **HTTPS** criptează traficul dintre browser și server: fără el, parolele și datele din formulare circulă în clar prin rețea. Local lucrăm pe HTTP, dar orice aplicație publicată folosește HTTPS, cu un certificat gratuit (de exemplu Let's Encrypt).

7. Salvarea lucrului și oprirea

```
git add .
git commit -m "Laboratorul 10: validare cu Symfony Validator, erori in formular"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

> **Push.** `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

8. Exerciții

1. Adăugați regula ca numele autorului să aibă cel puțin 2 caractere (`min` în `Assert\Length`, cu mesaj în română). Testați din Postman și din formular.
2. Afișați erorile și pentru câmpurile `rating` și `comment` și dați câmpurilor cu eroare un chenar roșu (atributul `aria-invalid` și o regulă CSS).
3. ★ (opțional) Adăugați o regulă care respinge comentariile ce conțin cuvântul „spam” (`Assert\Regex`, cu `match: false`).

9. Ce arătați la final

- răspunsul 400 din Postman cu cele trei erori și aceleași erori afișate în formular;
- testul XSS și răspunsul de la `/api/demo/parola`, cu explicația hash-ului;
- rezultatul comenzii `git check-ignore` și commit-ul „Laboratorul 10” pe GitHub, pe branch-ul vostru.

Resurse pentru exersare

- **W3Schools – SQL Injection:** https://www.w3schools.com/sql/sql_injection.asp
- **W3Schools – validarea formularelor în PHP:** https://www.w3schools.com/php/php_form_validation.asp
- **Tutorial PHP în limba română – introducere (video):** https://www.youtube.com/watch?v=JaTO46oiCEM
- **Symfony – Validation:** https://symfony.com/doc/current/validation.html
- **OWASP Top 10:** https://owasp.org/Top10/

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] OWASP Foundation, *OWASP Top 10*. owasp.org/Top10

[3] Symfony, „Validation” și „Configuring Symfony”, documentație oficială. symfony.com/doc/current

[4] OWASP Foundation, „Password Storage Cheat Sheet”. cheatsheetseries.owasp.org

[5] The PHP Group, „password_hash”, *PHP Manual*. php.net

[6] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*. Universitatea „Vasile Alecsandri” din Bacău, 2026.